

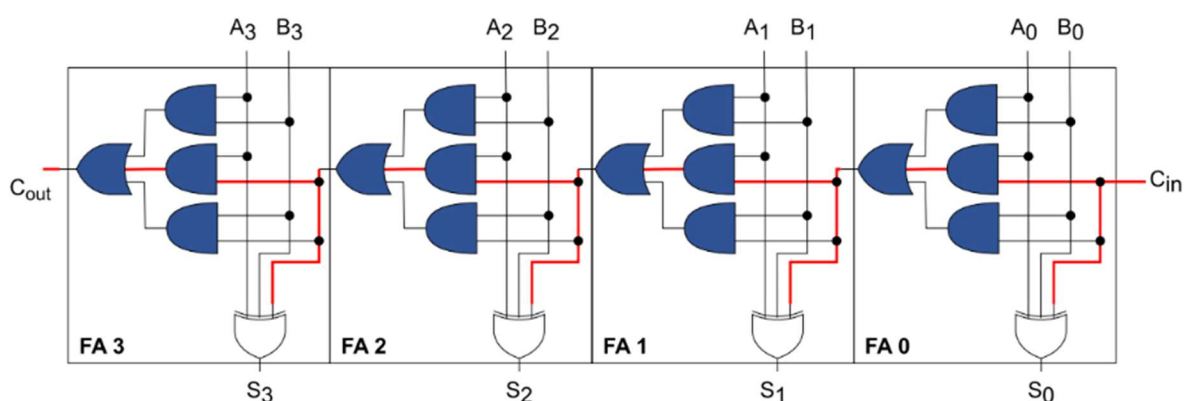
Báo Cáo:

Mạch Cộng Carry Lookahead

Họ và tên	MSSV
Huỳnh Phạm Minh Tú	22207094

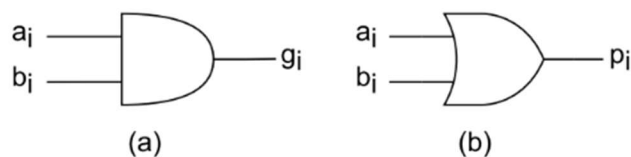
1. Kiến trúc mạch cộng Carry Lookahead (CLA):

Kiến trúc của mạch cộng Ripple Carry đơn giản nhưng gây ra độ trễ trong việc lan truyền tín hiệu nhớ (carry), cụ thể như được trình bày trong hình 4, khối FA sau phụ thuộc vào tín hiệu carry của khối FA trước, carry out (Cout) của FA 3 phụ thuộc vào carry out của FA 2, carry out của FA 2 phụ thuộc vào carry out của FA 1, carry out của FA 1 phụ thuộc vào carry out của FA 0, và carry out của FA 0 phụ thuộc vào carry in (Cin).



Hình 4. Minh họa độ trễ của việc lan truyền tín hiệu carry của mạch cộng Ripple Carry.

Mạch cộng Carry Lookahead là một giải pháp cho việc rút ngắn thời gian lan truyền tín hiệu carry. Carry Lookahead được xây dựng dựa trên hai thành phần cốt lõi là generate (gi) và propagate (pi). Thành phần generate là output của cổng AND hai input, trong khi đó, thành phần propagate là output của cổng OR hai input, được minh họa lần lượt trong hình 5.a và hình 5.b.



Hình 5. a) generate. b) propagate.

Từ hình 4 có thể thấy được carry của mỗi FA được tính như sau:

$$c_{i+1} = (a_i \cdot b_i) + (a_i \cdot c_i) + (b_i \cdot c_i) = (a_i \cdot b_i) + (a_i + b_i) \cdot c_i \quad (1)$$

Thay gi và pi vào biểu thức (1):

$$c_{i+1} = g_i + p_i \cdot c_i \quad (2)$$

Dựa trên biến đổi (2), chúng ta có thể biểu diễn carry của mạch cộng 4-bit như sau:

$$c1 = g0 + p0 \cdot c0$$

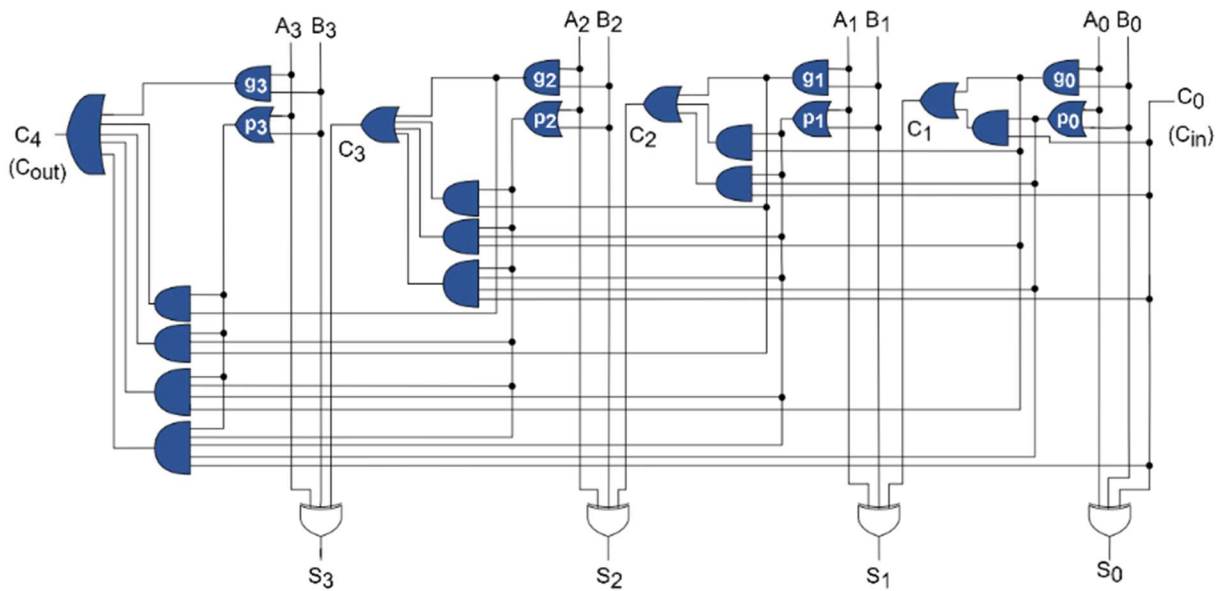
$$c2 = g1 + (p1 \cdot g0) + (p1 \cdot p0 \cdot c0)$$

$$c3 = g2 + (p2 \cdot g1) + (p2 \cdot p1 \cdot g0) + (p2 \cdot p1 \cdot p0 \cdot c0)$$

$$c4 = g3 + (p3 \cdot g2) + (p3 \cdot p2 \cdot g1) + (p3 \cdot p2 \cdot p1 \cdot g0) + (p3 \cdot p2 \cdot p1 \cdot p0 \cdot c0) \quad (3)$$

Lưu ý: $c0, c4$ tương ứng lần lượt với c_{in}, c_{out} trong hình 4.

Từ các biến đổi (3), kiến trúc đầy đủ của mạch cộng Carry Lookahead 4-bit được miêu tả trong hình 6.



Hình 6. Mạch cộng Carry Lookahead 4-bit.

Mặc dù kiến trúc mạch cho thấy, các bước tính toán do mạch CLA được thực hiện song song có tốc độ cao, phù hợp cả với số bit cao, được ứng dụng trong nhiều lĩnh vực như FPGA, DSP nhưng CLA cũng có những khuyết điểm lớn:

- Cần nhiều cổng logic, phức tạp và tốn nhiều diện tích.
- Tiêu thụ nhiều năng lượng.
- Không hiệu quả với hệ thống có số bit thấp.

Do đó, khi thực hiện tính toán cần chọn ra kiến trúc phù hợp với hệ thống nhằm mang lại hiệu suất cao nhất.

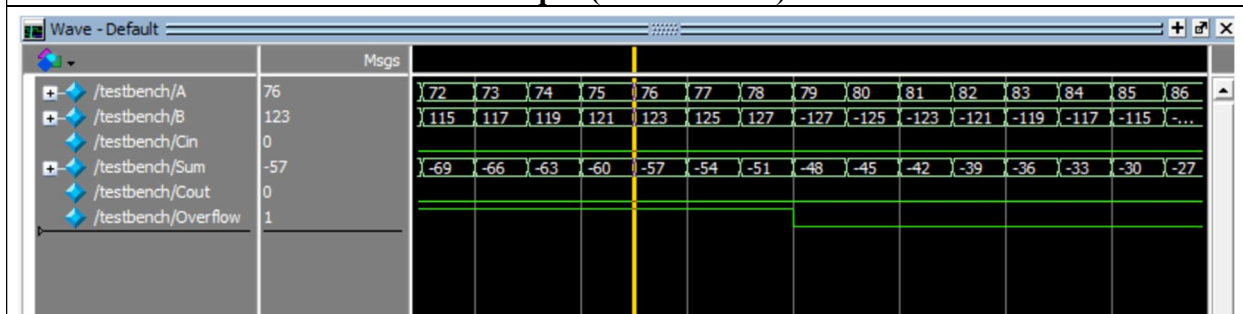
2. Thực hành

a. Mạch CLA 1 bit:

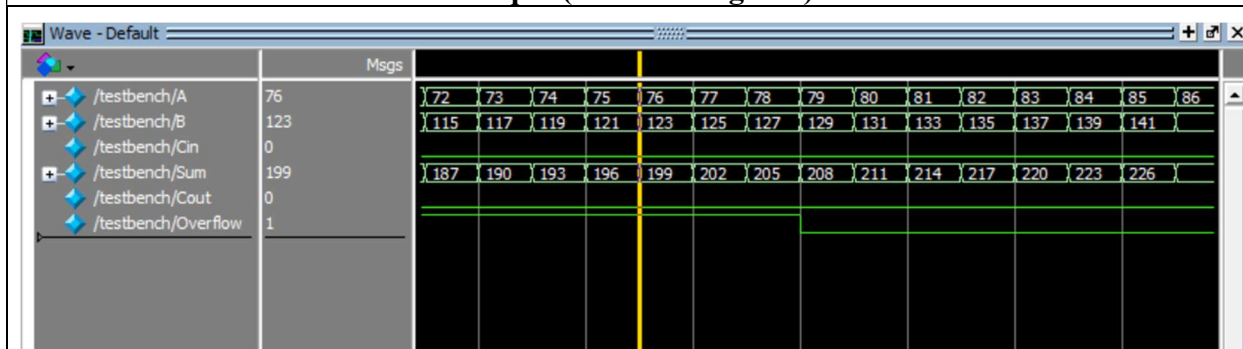
Code	
<pre> module lookahead_adder_1bit(output S, output Cout, input A, input B, input Cin); wire G, P; assign G = A & B; // A and B assign P = A ^ B; // A xor B assign Cout = G (P & Cin); // Cout = G or (P and Cin) assign S = A ^ B ^ Cin; // S = A xor B xor Cin (kết quả phép tính) endmodule </pre>	
Testbench	
<pre> module testbench1b; reg A, B, Cin; wire S, Cout; lookahead_adder_1bit uut (.A(A), .B(B), .Cin(Cin), .S(S), .Cout(Cout)); initial begin \$monitor("Time = %0t A = %b B = %b Cin = %b Sum = %b Cout = %b", \$time, A, B, Cin, S, Cout); A = 0; B = 0; Cin = 0; #10; A = 0; B = 0; Cin = 1; #10; A = 0; B = 1; Cin = 0; #10; A = 0; B = 1; Cin = 1; #10; A = 1; B = 0; Cin = 0; #10; A = 1; B = 0; Cin = 1; #10; A = 1; B = 1; Cin = 0; #10; A = 1; B = 1; Cin = 1; #10; \$finish; // Kết thúc mô phỏng end endmodule </pre>	
Kết quả	
	

b. Mạch CLA 8bit:

Code
<pre> module lookahead_adder_8bit(output [7:0] S, output Cout, output Overflow, // thêm cờ báo khi phép tính bị tràn input [7:0] A, B, input Cin); wire [7:0] G, P; wire [8:0] C; assign G = A & B; assign P = A ^ B; assign C[0] = Cin; assign C[1] = G[0] (P[0] & C[0]); assign C[2] = G[1] (P[1] & C[1]); assign C[3] = G[2] (P[2] & C[2]); assign C[4] = G[3] (P[3] & C[3]); assign C[5] = G[4] (P[4] & C[4]); assign C[6] = G[5] (P[5] & C[5]); assign C[7] = G[6] (P[6] & C[6]); assign C[8] = G[7] (P[7] & C[7]); assign Cout = C[8]; assign S = P ^ C[7:0]; assign Overflow = (A[7] == B[7]) && (S[7] != A[7]); // tính cờ tràn endmodule </pre>
Testbench
<pre> module testbench; reg [7:0] A, B; reg Cin; wire [7:0] Sum; wire Cout, Overflow; lookahead_adder_8bit uut (.A(A), .B(B), .Cin(Cin), .S(Sum), .Cout(Cout), .Overflow(Overflow)); initial begin A = 8'd53; B = 8'd77; Cin = 0; \$monitor("Time = %0t A = %d B = %d Cin = %b Sum = %d Cout = %b Overflow = %b", \$time, A, B, Cin, Sum, Cout, Overflow); end always #10 begin A = A + 8'b00000001; B = B + 8'b00000010; end endmodule </pre>

Kết quả (với số có dấu)

Với số có dấu, khi cờ tràn = 1, các phép tính cho ra kết quả sai, không chính xác do bị tràn bit, khi cờ tràn = 0, các phép tính cho ra kết quả chính xác.

Kết quả (với số không dấu)

Với số không dấu, tất cả các phép toán đều chính xác kể cả khi cờ tràn = 1